



2.10 Библиотеки

Привет!

На связи шпаргалка урока 2.10 Библиотеки, в котором вы найдете материалы по теме урока.

Для ускорения разработки в Java применяются библиотеки. Таких файлов, библиотек, можно найти очень большое количество. В данном уроке мы познакомимся с ключевыми особенностями библиотек, базовыми наборами библиотек и способами их установки.

Время прочтения: 40 минут

Что такое библиотеки?

Подключение библиотек

Создание проекта

Подключение зависимости без системы сборки

Подключение зависимости через Maven/Gradle

Репозитории библиотек

Maven Central и другие репозитории

Поиск и загрузка библиотек

Конфликты версий

Maven BOM

Использование библиотеки в проекте

Класс StringUtils из commons-lang3.

Что такое библиотеки?

Библиотеки являются неотъемлемой частью разработки на Java и любом другом языке. Библиотеки в Java — набор классов, позволяющих упростить и ускорить работу разработчика в написании шаблонного кода.

Допустим, вы работаете в мобильном операторе. Тарифы, которые вы предлагаете клиентам, имеют большой пакет услуг. Например, одна из услуг позволяет клиентам при звонке внутри вашей сети не тратить оплаченные минуты. Другая услуга позволяет переводить минуты другому абоненту вашей сети.

С точки зрения архитектуры, эти две услуги, вероятно, работали бы с помощью разных сервисов (приложений). Но у этих сервисов есть нечто общее. Они обязаны перед выполнением логики проверить, что взаимодействие идет между абонентами именно вашего оператора.

Абстрагируемся от возможностей перехода к вам с сохранением номера другого оператора и прочих условностей реальной жизни. Предположим, что наш вымышленный оператор имеет пул номеров с кодом 999. Как можно проверить принадлежность абонента? Например, нам бы пришлось писать код, который «разбивает» номер телефона на части и проверяет, что первые 3 цифры соответствуют нашему коду 999. Но если такого рода услуг не две, а десяток, то получается, что все 10 приложений должны иметь внутри себя эту проверку.

Как это можно реализовать?

Решений несколько.

В первом случае вы можете написать отдельный сервис (приложение), развернуть его где-то в облаке и каждый ваш сервис, обслуживающий услугу, будет отправлять через интернет запрос к проверяющему абонента сервису. Главной проблемой в данном случае будет то, что этот функционал слишком мал для выноса в отдельное приложение.

Другое решение еще хуже. Мы можем просто скопировать и вставить код проверки в каждое нуждающееся приложение. Но здесь мы сталкиваемся с еще одной проблемой. Если наша клиентская база расширится и нам потребуется ввести новый код (например, 998), мы будем обязаны «залезть» в код каждого из ранее написанных приложений по услугам и добавить в проверку новый код. После этого, безусловно, каждое приложение будет обязано пройти тестирование и прочие процедуры, связанные с размещением в «боевой» среде.

Именно для таких «регулярных» задач и существуют библиотеки.

Библиотеки — это архив скомпилированных классов, которые добавляются в ваш проект и позволяют вам взаимодействовать с ними стандартным способом через создание объектов и вызовов их методов.

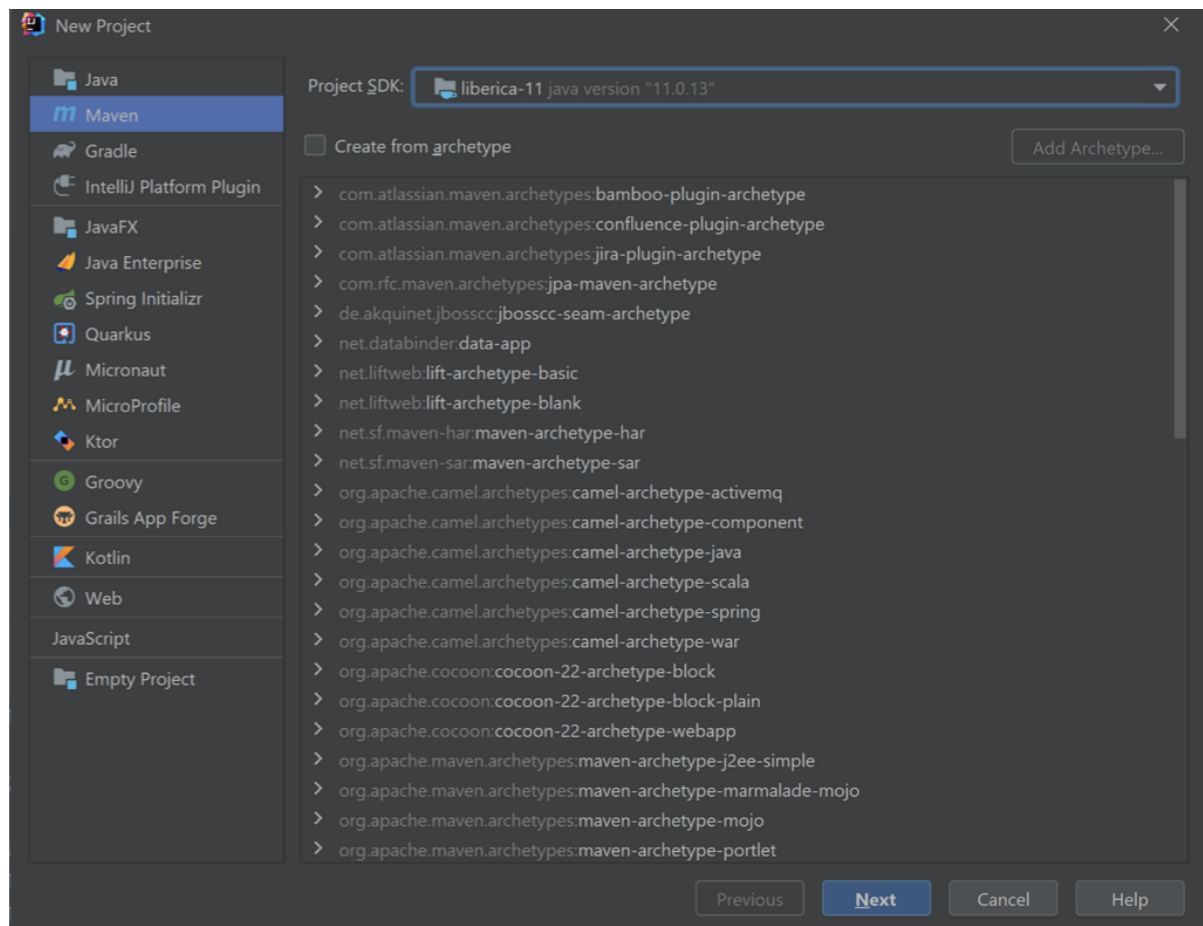
Этот архив классов при сборке вашего приложения укладывается «рядом» с вашими классами, образуя уже запускаемый JAR-архив, который JVM имеет возможность выполнить.

Обычно за добавление библиотек отвечают системы сборки (Maven/Gradle), но мы сегодня рассмотрим добавление зависимости через IDEA, так как это позволит в полной мере оценить преимущества систем сборки.

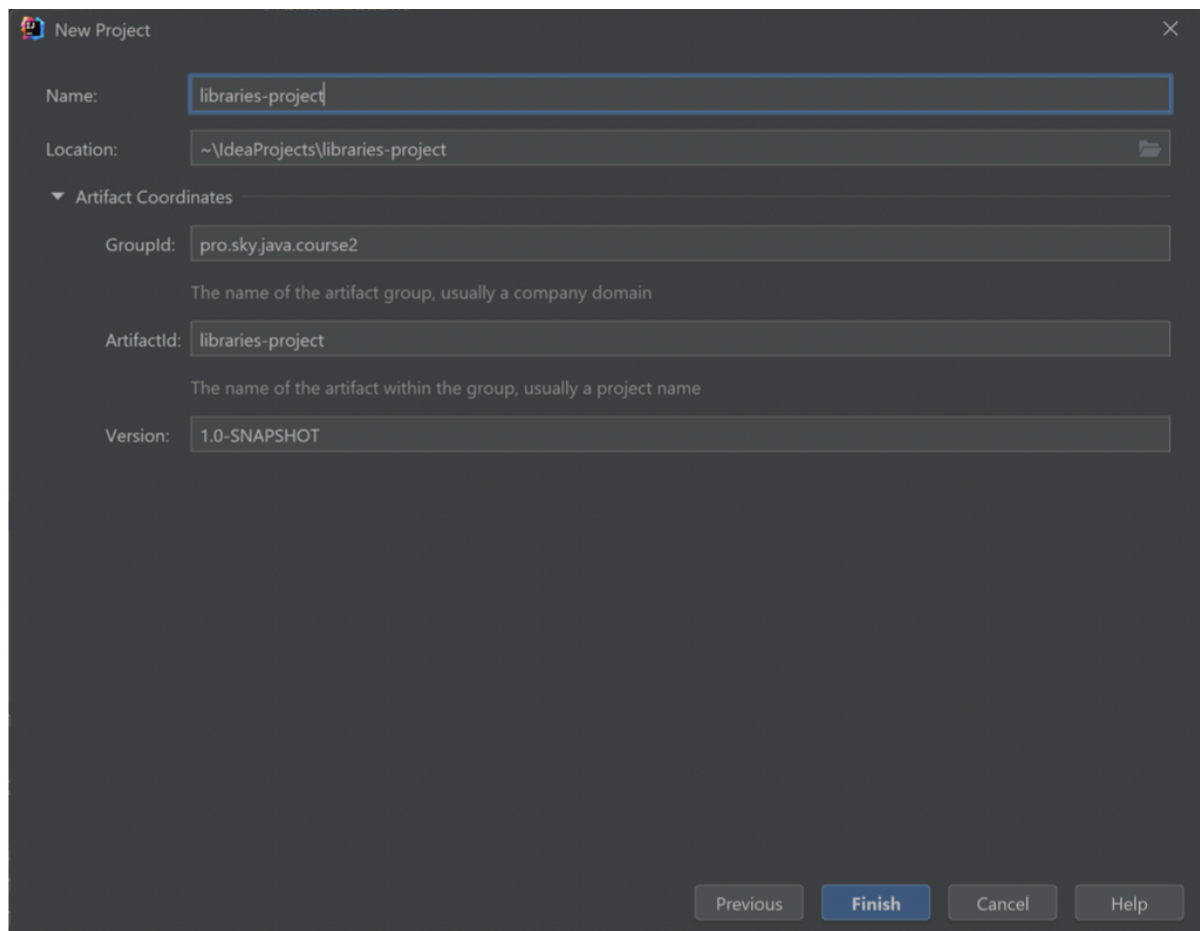
Подключение библиотек

Создание проекта

Шаг 1. Для подключения библиотек нам понадобится создать новый проект. Сегодняшний наш проект никак не связан со Spring, потому создавать его уже привычным способом через Spring Initializr мы не будем. Однако Maven нам нужен. В связи с этим необходимо выбрать в меню создания проекта пункт Maven.

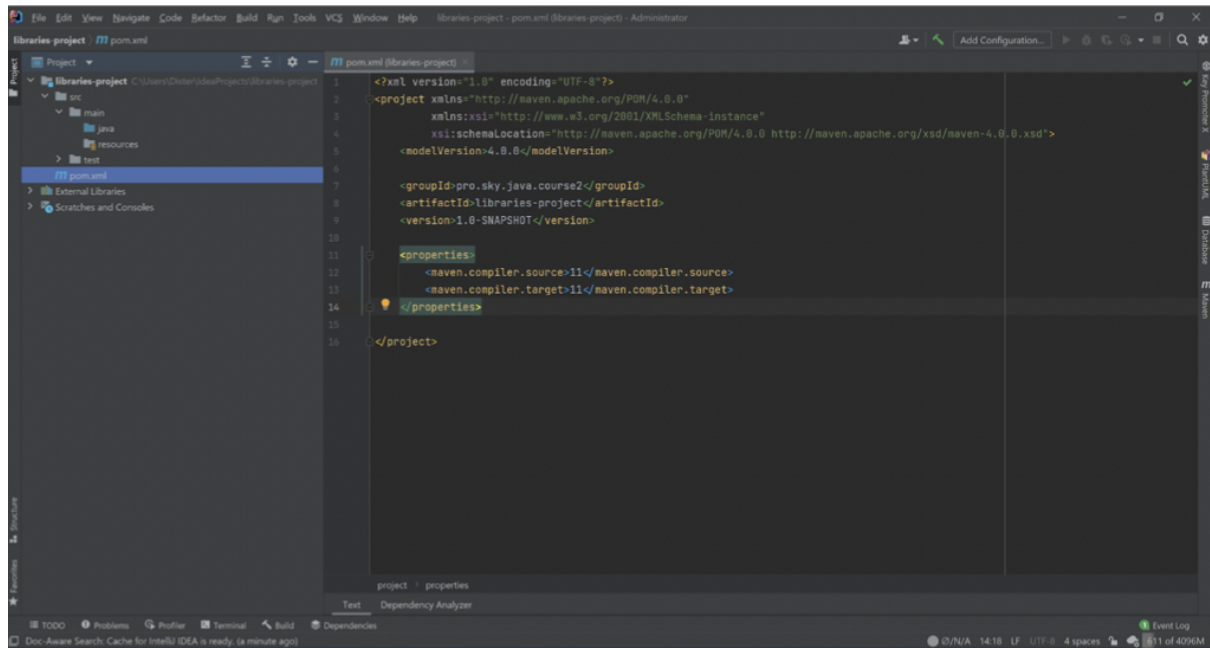


Шаг 2. Далее нажимаем Next и попадаем на следующее окно. В нем нам необходимо заполнить уже знакомые имена проекта, основного пакета и выбрать расположение проекта на вашем компьютере.



Шаг 3. Заполняем, нажимаем Finish и попадаем в IDEA.

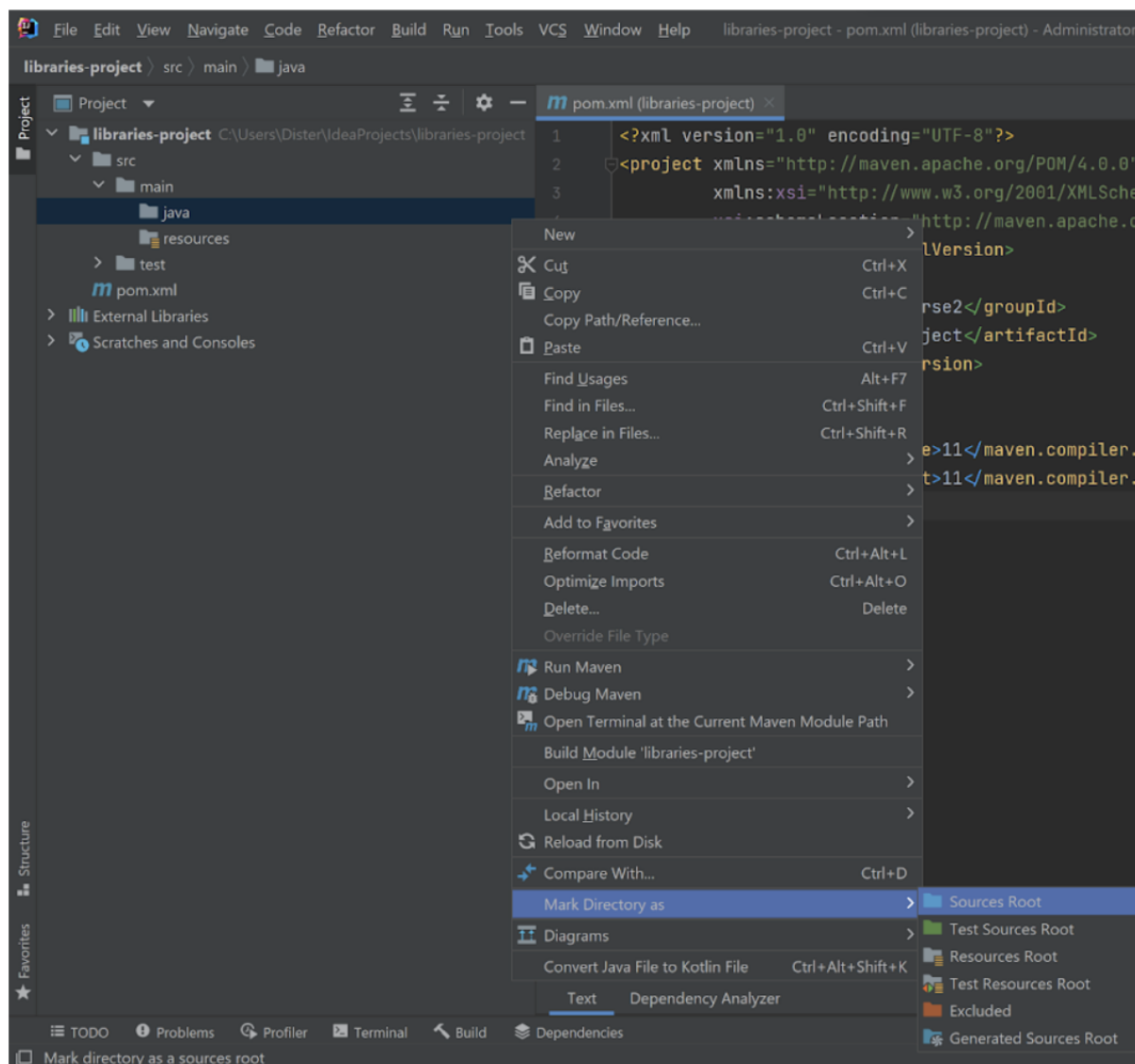
Наш проект создан, и перед нами открылся pom.xml стандартного содержания. В данном случае он выглядит следующим образом:



Шаг 4. Убедитесь, что в левом окне структура проекта выглядит таким же образом.

Если отсутствует src, main или java, их нужно будет создать вручную.

Если папка java не горит синим, ее потребуется создать (если не создана), затем нажать на нее правой кнопкой мыши -> Mark Directory as -> Sources Root.



Только после этого шага IDEA понимает, что в этой папке находятся «исходники» и их нужно расценивать не как обычные текстовые файлы, а как файлы с кодом.

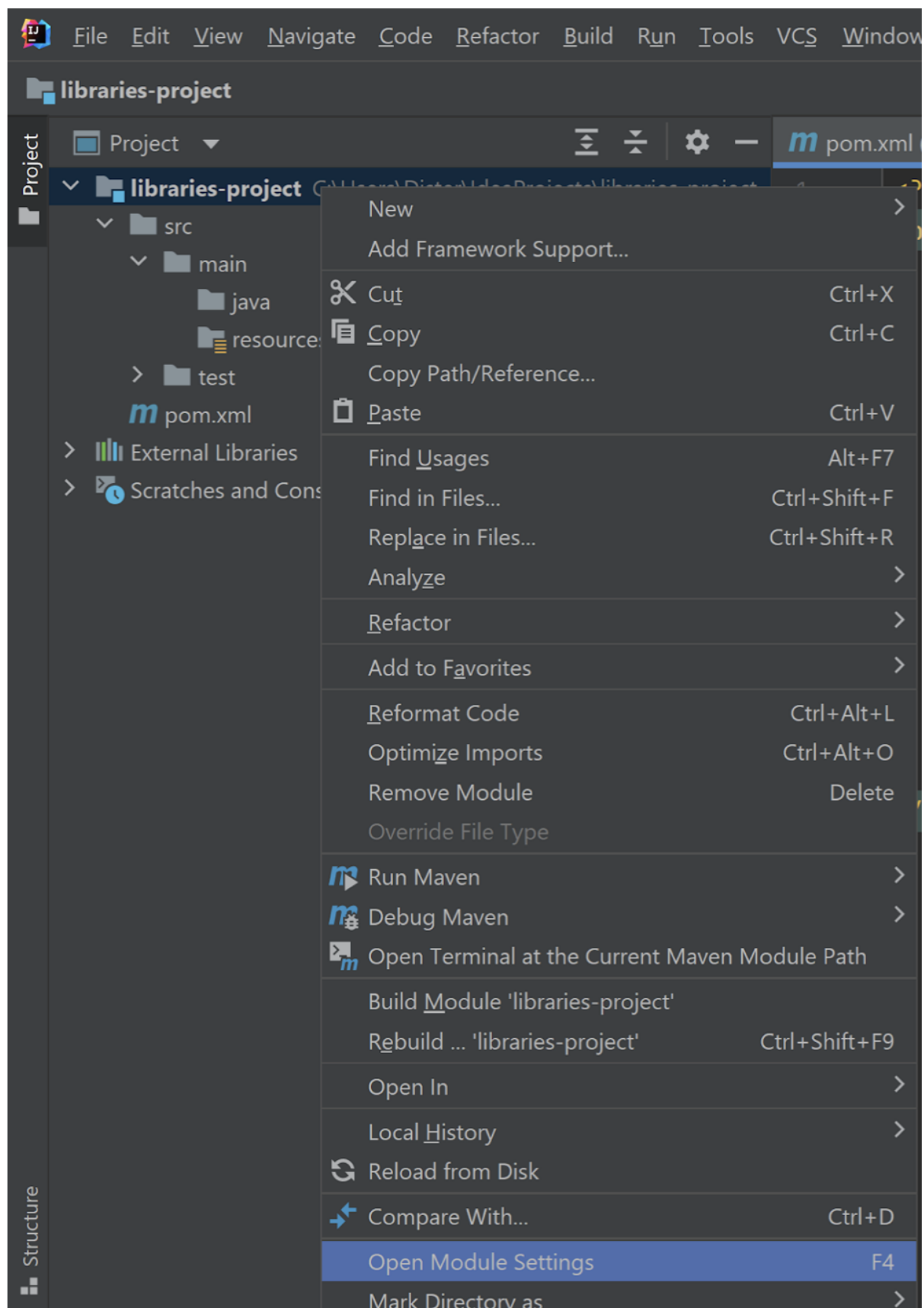
Подключение зависимости без системы сборки

Теперь перейдем к подключению библиотеки стандартными средствами IDEA. Для этого нам первым делом нужно выкачать *jar*-файл (архив) библиотеки с классами.

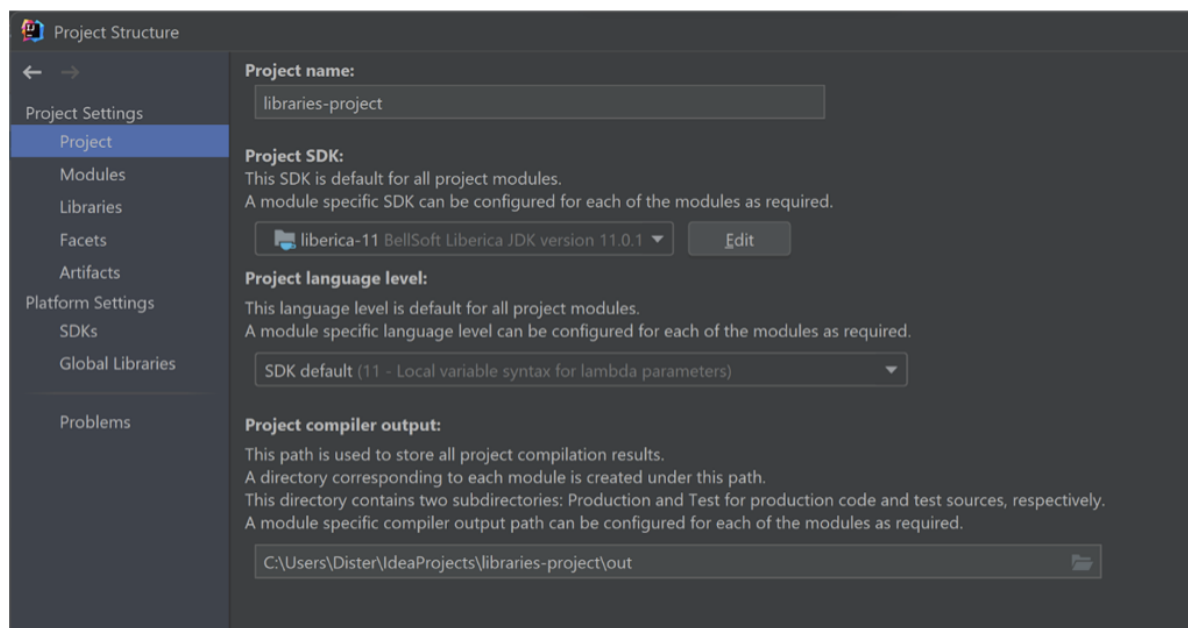
Достать эти файлы нужно из определенных хранилищ, куда авторы библиотек выкладывают новые версии своих работ. О подобных хранилищах

мы поговорим немного позже. Пока представим, что jar-файл у нас есть.

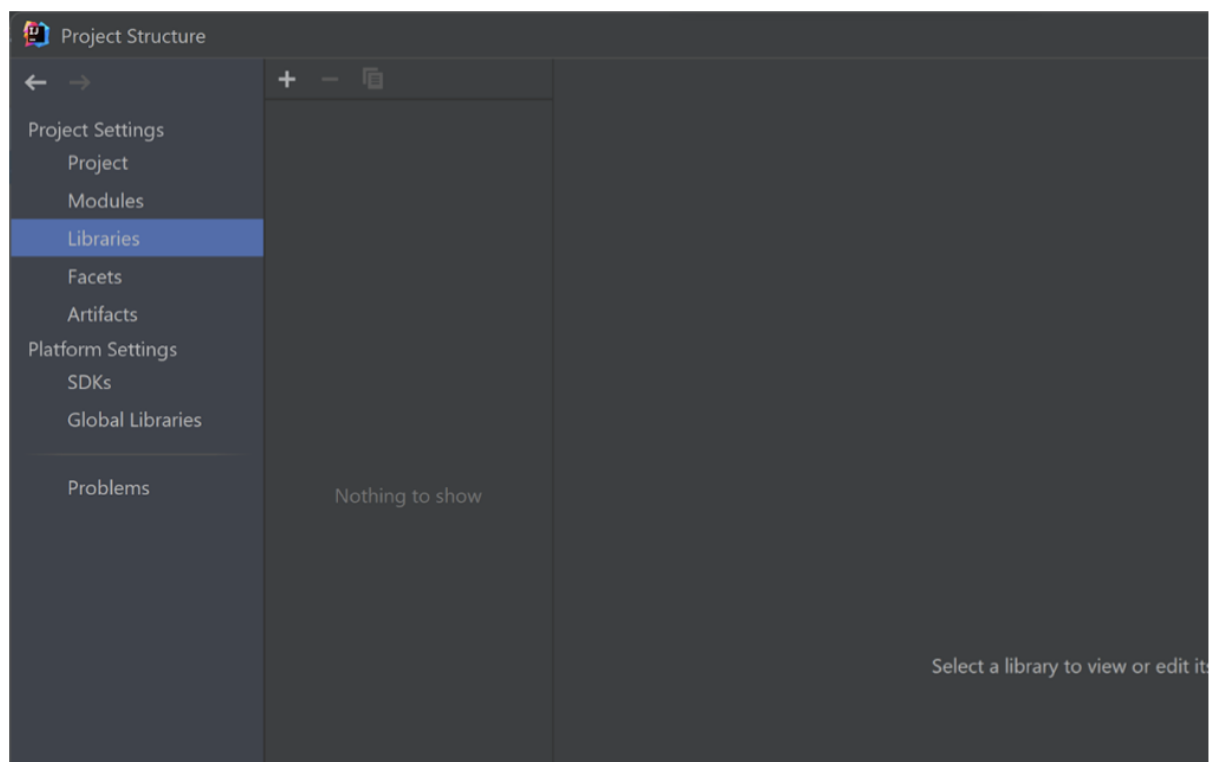
Шаг 1. Для добавления библиотеки в проект без системы сборки нам необходимо нажать «Перейти» в настройки проекта. Кликаем правой кнопкой мыши на названии нашего проекта в окне Project в левом верхнем углу и выбираем пункт Open Module Settings.



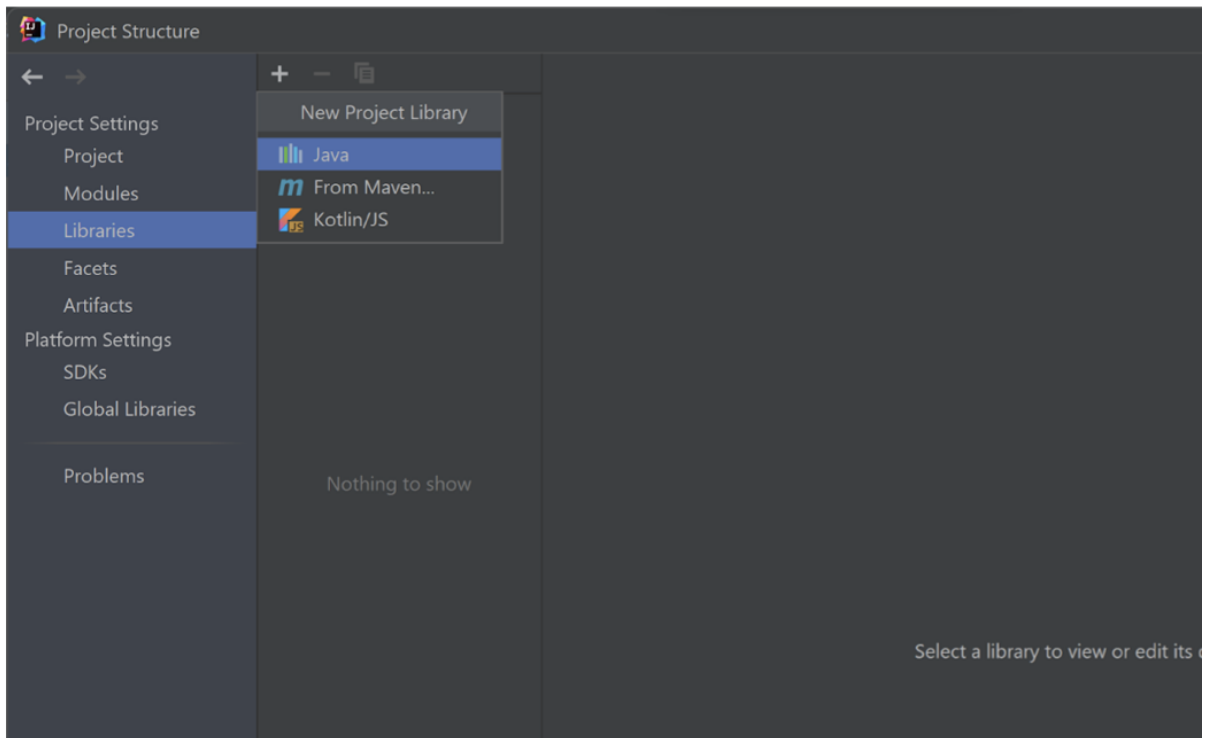
Открывается следующее окно:



Шаг 2. В левой части выбираем раздел Libraries.

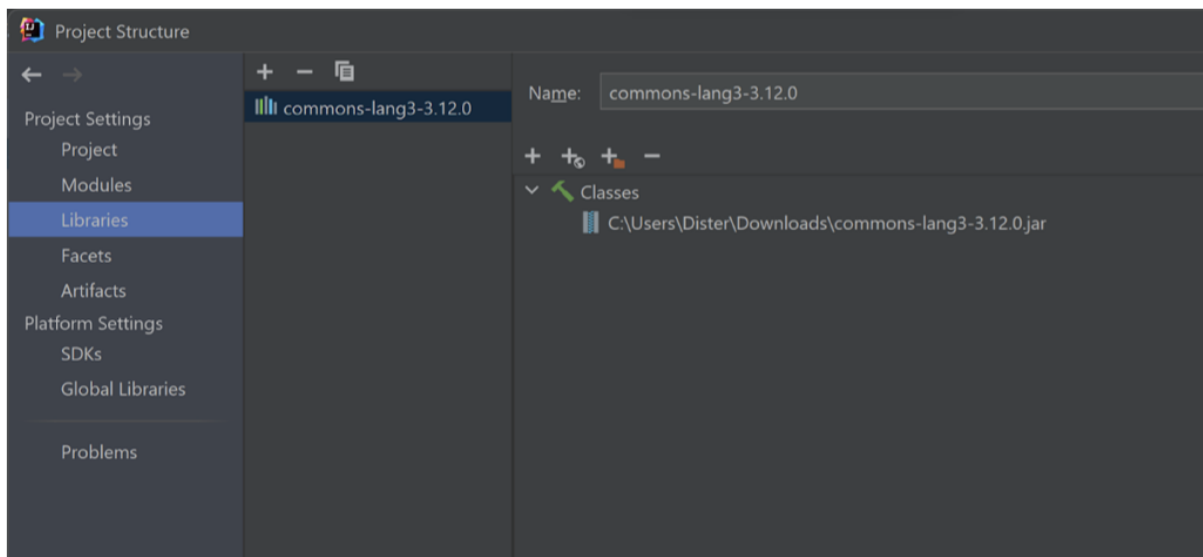


Шаг 3. В центре сверху нажимаем на «+» и выбираем Java.



Шаг 4. Далее открывается файловый менеджер, находим файл .jar с библиотекой на нашем компьютере и нажимаем Ok.

Теперь в нашем окне Libraries появилась добавленная библиотека.

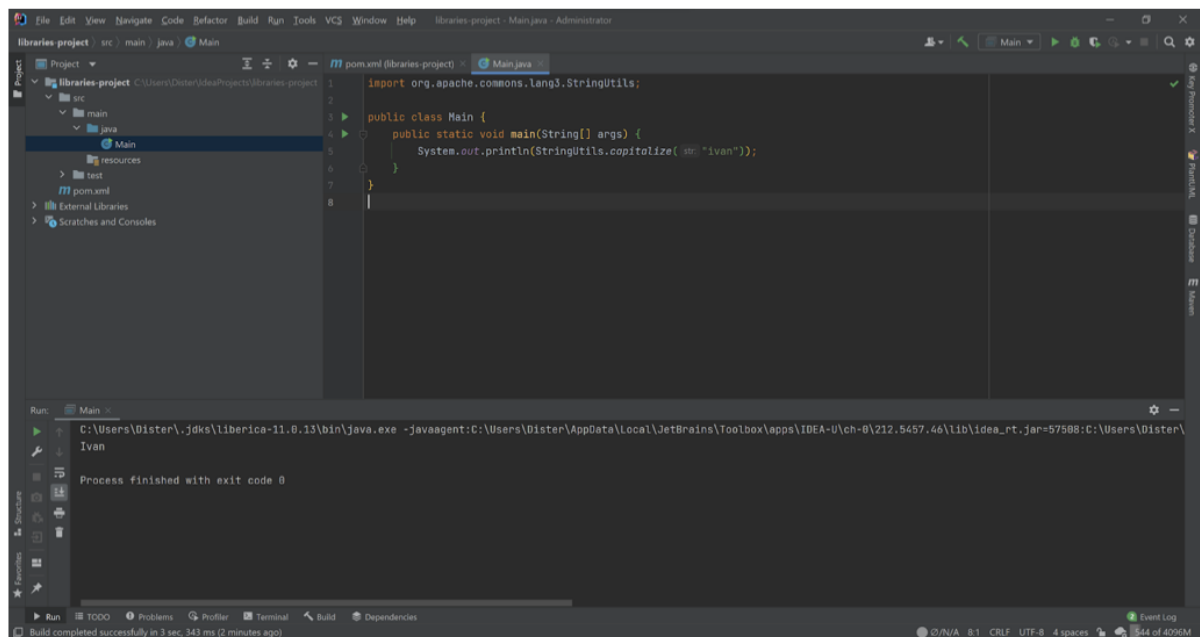


Это значит, что библиотека была успешно добавлена.

Шаг 5. Нажимаем Ok и возвращаемся в проект.

Проверим, что библиотека добавилась.

Шаг 6. Создадим класс Main, в нем объявим метод main и попробуем вызвать метод класса StringUtils из подключенной библиотеки commons-lang.



Как видите, библиотека была добавлена. Обратите внимание на `import` класса `StringUtils`. Он происходит из пакета `commons.lang3`, что соответствует подключенной нами библиотеке.

Проделывать проверку библиотек указанным выше способом обычно не требуется. Данный способ приводится исключительно в ознакомительных целях. В ваших проектах не должно быть библиотек, добавленных таким способом.

Способ работает, и мы в этом убедились, но что будет, если нам нужно в проекте иметь не 1, 2 или 5 библиотек, а десятки, как в выполненных ранее проектах на Spring, ведь в нем собраны десятки, а то и сотни различных библиотек.

Именно здесь в бой вступают системы сборки **Maven** и **Gradle**. Их функционал практически неограничен, но мы сегодня коснемся самых базовых вещей, а именно добавление зависимостей в проект.

Подключение зависимости через Maven/Gradle

Вернемся к упомянутому ранее `pom.xml`.



Мы разбирали структуру файла `pom.xml` на уроке 2.3, когда создавали впервые Spring Boot-проект. Если в памяти это не отложилось, пересмотрите материалы урока 2.3.

Сейчас разберем только блок зависимостей. В нашем созданном проекте `pom.xml` не содержит каких-либо намеков на зависимости. Чтобы начать добавлять библиотеки, нам нужно создать в `pom.xml` блок зависимостей. В этом нам поможет IDEA.

Шаг 1. Делаем отступ от того, что написано в `pom.xml`, и открываем тег `<dependencies>`.

IDEA уже на первых буквах нам подскажет. Останется только согласиться с ней и нажать `Enter`.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org
5     <modelVersion>4.0.0</modelVersion>
6
7     <groupId>pro.sky.java.course2</groupId>
8     <artifactId>libraries-project</artifactId>
9     <version>1.0-SNAPSHOT</version>
10
11     <properties>
12         <maven.compiler.source>11</maven.compiler.source>
13         <maven.compiler.target>11</maven.compiler.target>
14     </properties>
15
16     <dep
17         dependencies http://maven.apache.org/POM/4.0.0
18         dependencyManagement http://maven.apache.org/POM/4.0.0
```

Блок создан. Обратите внимание, что все блоки должны быть написаны после открывающего тега `<project>` и до закрывающего тега `</project>`.

```

15
16     <dependencies>
17         |
18     </dependencies>
19
20 </project>

```

Шаг 2. Теперь нужно добавить тег `<dependency>` и прописать внутри него данные библиотеки.

В данных должен быть пакет, который автор указывал при создании проекта, название проекта и версия.

```

<dependency>
<groupId>org.apache.commons</groupId>
<artifactId>commons-lang3</artifactId>
<version>3.12.0</version>
</dependency>

```

Для нашей библиотеке `commons-lang3` блок должен выглядеть следующим образом:

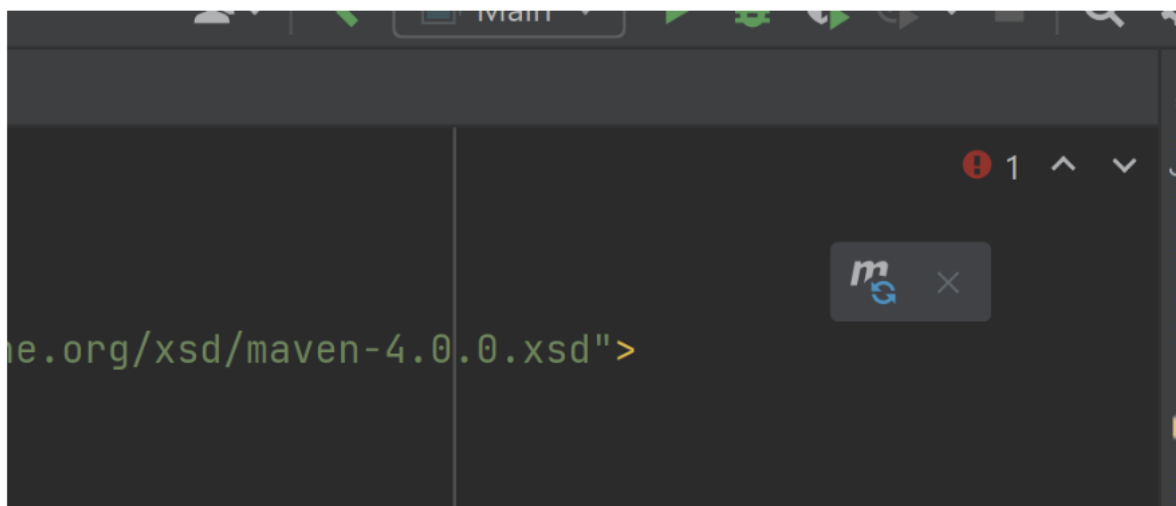
```

15
16     <dependencies>
17         <dependency>
18             <groupId>org.apache.commons</groupId>
19             <artifactId>commons-lang3</artifactId>
20             <version>3.12.0</version>
21         </dependency>
22     </dependencies>
23

```

Шаг 3. После добавления блока `<dependency>` Maven попросит вас синхронизироваться с `pom.xml`.

За это отвечает следующая иконка в правом верхнем углу.



Шаг 4. Нажмите ее — и Maven начнет обновлять проект под добавленные вами теги. На этом всё. Библиотека добавлена в ваш проект.

Если ваш проект работает с помощью системы сборки Gradle, там блок зависимостей и сами зависимости выглядят короче и лаконичнее.

Структуру конфигурационного файла `build.gradle` мы также разбирали на уроке 2.3.

```
dependencies {  
    implementation 'org.apache.commons:commons-lang3:3.12.0'  
}
```

Вспоминая о проблеме сотен библиотек и библиотек, которым нужны другие библиотеки для корректной работы, стоит добавить, что при добавлении библиотеки через систему сборки система сборки анализирует зависимости этой библиотеки и, если ей нужны еще какие-то библиотеки для работы, загружает и их.

Именно поэтому, когда вы создавали Spring-проект, ваши зависимости можно было пересчитать по пальцам, а по факту, «под капотом», было подгружено несколько десятков, а то и сотен различных библиотек.

Системы сборки избавляют нас от необходимости самостоятельно следить за тем, чтобы все необходимые зависимости подгрузились.

Репозитории библиотек

Maven Central и другие репозитории

Мы разобрались в способах подключения библиотек, но откуда эти библиотеки загружаются?

В интернете есть несколько сайтов, которые называются «репозитории». Эти сайты являются хранилищами для jar-файлов библиотек, фреймворков и т. д.

Главный из них и самый популярный называется Maven Central.

Располагается он по адресу <https://search.maven.org/> и указан по умолчанию во всех системах сборки. В pom.xml этот репозиторий указан неявно, если отсутствует блок <repositories>. В build.gradle же этот блок присутствует всегда и в него по умолчанию IDEA добавляет тот самый mavenCentral.

```
repositories {  
    mavenCentral()  
}
```

В мире существуют и другие репозитории. Например, **JCenter** и **Bintray**. Однако по популярности им не сравниться с Maven Central. Официальные андроид-библиотеки хранятся в специальном публичном репозитории компании **Google**.

Большинство компаний имеют свои приватные репозитории для загрузки и выгрузки корпоративных библиотек. В зависимости от места работы ваш блок repositories в pom.xml или build.gradle будет выглядеть по-разному.

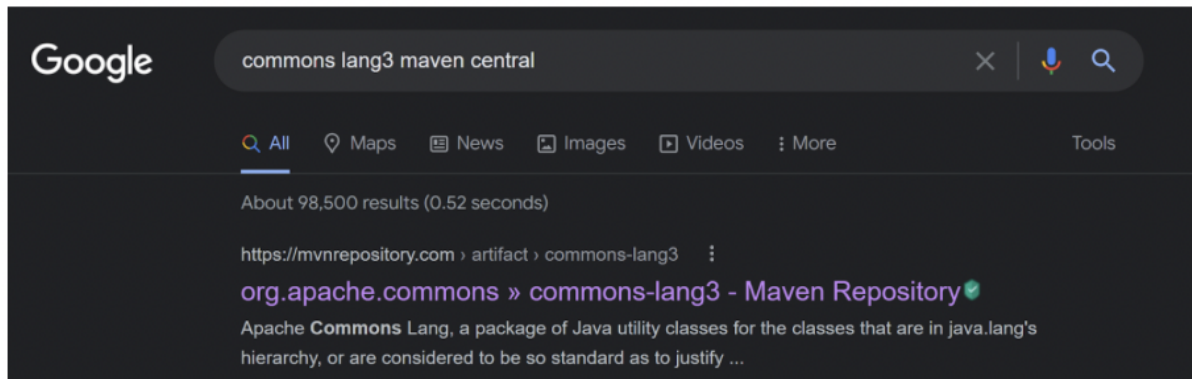
Поиск и загрузка библиотек

Мы уже разобрались в том, как подключить библиотеку и откуда они выкачиваются, но как же найти и загрузить непосредственно ту библиотеку, что нам нужна в огромном хранилище Maven Central?

Нет ничего проще. Если вам известно название библиотеки, скорее всего Google или любой другой поисковик первой ссылкой выдаст корректный адрес на библиотеку. Или же вы можете вручную зайти на указанный ранее адрес

Central и вбить там в поиске название. Попробуем найти упомянутую ранее **commons-lang3**.

Шаг 1. Для этого пишем запрос: commons lang3 maven central.



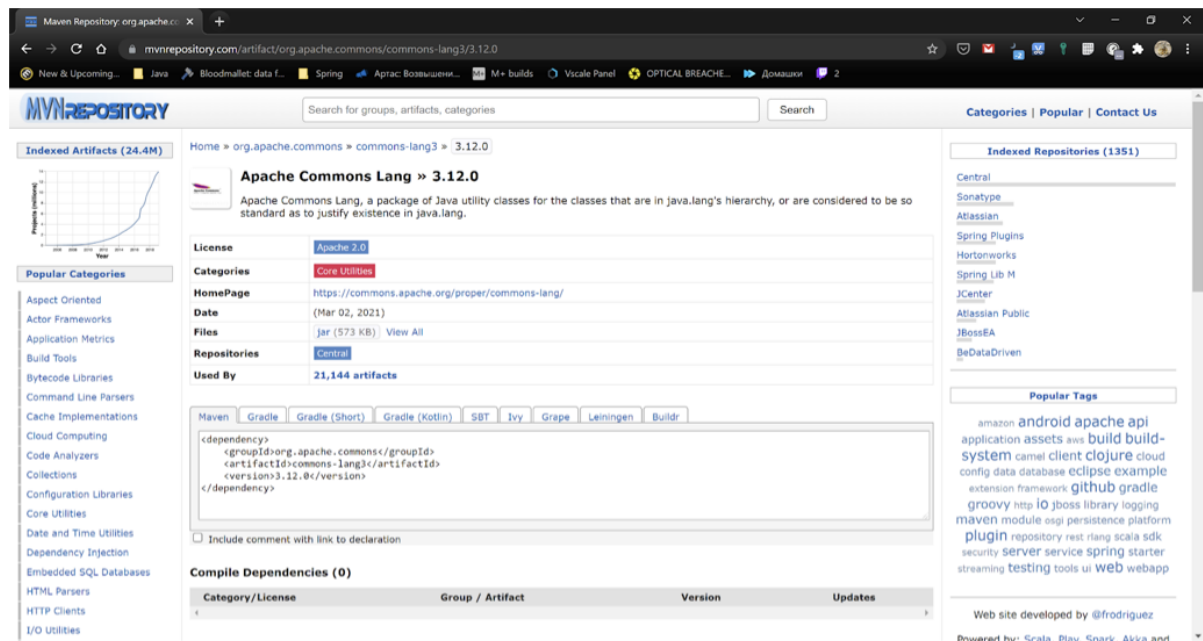
Задача выполнена. Первой же ссылкой нас перенаправляет на *mvnrepository.com*. Это тоже один из адресов того самого Maven Central.

Шаг 2. При нажатии попадаем на следующую страницу:

Version	Repository	Usages	Date
3.12.x	Central	2,245	Mar, 2021
3.11.x	Central	3,104	Jul, 2020
3.10.x	Central	1,932	Mar, 2020
3.9.x	Central	4,002	Apr, 2019
3.8.x	Central	3,626	Sep, 2018
3.7.x	Central	1,158	Aug, 2018
3.6.x	Central	3,214	Nov, 2017
3.5.x	Central	2,256	Jun, 2017
3.4.x	Central	2,613	Oct, 2016
3.3.2	Central	4,672	Apr, 2015
3.3.2	Central	2,684	Apr, 2014

На странице видим много информации. Здесь и название, и описание, и лицензионная информация, и наличие библиотеки в других репозиториях (вкладки над списком версий). Нас интересует только список версий. По списку версий можно понять, насколько жив проект, какая версия наиболее популярна и когда было последнее обновление.

Шаг 3. Выбираем последнюю версию (на момент написания это была 3.12.0) и переходим на страницу этой версии.



Шаг 4. При открытии страницы нас встречает множество вкладок под различные системы сборки. Выбираем вкладку Maven и копируем текст из открывшейся вкладки. Этот текст и является тем самым блоком `<dependency>`. Выделяем его, копируем и вставляем в блок `<dependencies>` нашего `pom.xml` за предыдущими блоками `<dependency>`.

Например, в случае добавления двух библиотек блок `<dependencies>` должен выглядеть так:

Конфликты версий

В разработке бывают ситуации, когда версия библиотеки конфликтует с версиями своих зависимостей. Или даже несколько библиотек используют одну и ту же зависимость, но разной версии.

В такой ситуации система сборки попытается самостоятельно разрешить конфликт, загрузив ту версию, что является последней, но это решение не всегда рабочее, так как новая версия зависимости может быть написана

совершенно другими людьми и код в ней идет по другому пути, возвращая отличный от ожидаемого результат.

В ситуациях, когда это критично, компании вводят отдельный `pom.xml`, который содержит намертво прибитые номера версий для отдельных библиотек. Этот `pom.xml` отличается от того, что вы видите при создании проекта. Его основная и единственная задача — в хранении списка версий для отдельных зависимостей. Называется этот `pom.xml` аббревиатурой BOM, т. е. Bill Of Materials.

BOM — это особый вид `pom.xml`, используемый для управления версиями зависимостей проекта и обеспечения центрального места для определения и обновления этих версий.

Maven BOM

Вы уже сталкивались с BOM, когда создавали Spring-проект через Spring Initializr.

Он хранится в блоке `<parent>` вашего `pom.xml`.

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.5.6</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>
```

Если зажать Ctrl и кликнуть на `spring-boot-starter-parent`, вы попадете в этот самый `pom.xml`.

В нем будет много текста с указанием версий непосредственно плагина Spring Boot, информацией о разработчиках и т. д., а также будет еще один блок `<parent>`.

```

<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-dependencies</artifactId>
  <version>2.5.6</version>
</parent>

```

Здесь уже начинается самое интересное.

Если мы провалимся в этот самый spring-boot-dependencies, то увидим pom.xml на 3000 строк.

```

2895         <groupId>org.codehaus.mojo</groupId>
2896         <artifactId>xml-maven-plugin</artifactId>
2897         <version>${xml-maven-plugin.version}</version>
2898     </plugin>
2899 </plugins>
2900 </pluginManagement>
2901 </build>
2902 </project>
2903

```

Этот pom.xml состоит из двух частей.

В первой части у нас уже знакомые по разделам выше блоки <dependency>.

```

<dependency>
  <groupId>org.springframework.amqp</groupId>
  <artifactId>spring-rabbit</artifactId>
  <version>${spring-amqp.version}</version>
</dependency>

```

Заметьте, что в теге <version> стоит ссылка на переменную. Эти переменные собраны во второй части этого pom.xml. И именно по данным из этих двух pom.xml ваше Spring-приложение узнает, какие именно версии ему нужны для работы.

```
<javax.websocket.version>1.1</javax.websocket.version>  
<jaxen.version>1.2.0</jaxen.version>  
<jaybird.version>4.0.4.java8</jaybird.version>
```

Как видите, версии в Spring и Spring Boot намертво прибиты и никакого конфликта быть не может. Вот мы и познакомились с понятием BOM.

Использование библиотеки в проекте

Добавим в наш проект две библиотеки, упомянутые выше, а именно: commons-lang3 и guava. Блок <dependencies> должен выглядеть так:

```
<dependencies>  
<dependency>  
<groupId>org.apache.commons</groupId>  
<artifactId>commons-lang3</artifactId>  
<version>3.12.0</version>  
</dependency>  
<dependency>  
<groupId>com.google.guava</groupId>  
<artifactId>guava</artifactId>  
<version>31.0.1-jre</version>  
</dependency>  
</dependencies>
```

Рассмотрим применение этих двух библиотек в коде.

Класс StringUtils из commons-lang3.

Облегчает работу со строками, а также имеет массу проверок. Например, позволяет отрезать фрагмент строки после или до первого символа/последовательности символов. Или проверить, что строка полностью состоит из чисел/букв/пробелов.

```

public static void main(String[] args) {
    String lettersAndSpaceStr = "ivanov ivan";
    String emptyStr = " ";

    System.out.println(StringUtils.capitalize(str: lettersAndSpaceStr));
    System.out.println(StringUtils.deleteWhitespace(str: lettersAndSpaceStr));
    System.out.println(StringUtils.substringAfterLast(str: lettersAndSpaceStr, separator: " "));

    System.out.println(StringUtils.isAllUpperCase(cs: lettersAndSpaceStr));
    System.out.println(StringUtils.isAlphanumericSpace(cs: lettersAndSpaceStr));
    System.out.println(StringUtils.isWhitespace(cs: emptyStr));
}

```

Позволяет находить минимум и максимум среди любого количества переменных, а также удобно конвертировать строку в любой из числовых типов.

```

public static void main(String[] args) {
    int a = 5;
    int b = 100500;
    int c = Integer.MAX_VALUE;

    System.out.println(NumberUtils.min(a: a, b: b, c: c));
    System.out.println(NumberUtils.max(a: a, b: b, c: c));

    String floatedPointString = "1.0f";
    float f = NumberUtils.toFloat(str: floatedPointString);
    System.out.println(f);
}

```

Классы пакета `com.google.common.collect` из `guava` предлагают нам альтернативный способ создания коллекций благодаря классам-конструкторам `Lists`, `Sets` и `Maps`.

```
public static void main(String[] args) {  
    List<Integer> numbers = Lists.newArrayList( ...elements: 1, 2, 3);  
    List<Integer> numbers2 = Lists.newLinkedList( elements: numbers);  
  
    Set<Integer> numbersSet = Sets.newHashSet( ...elements: 5, 4, 3, 1, 1, 3, 4, 5);  
  
    System.out.println(numbers);  
    System.out.println(numbers2);  
    System.out.println(numbersSet);  
}
```

Как видите, в данном примере используются статические методы, а не создание через `new`.

Эти статические методы позволяют инициализировать коллекцию напрямую передачей в скобки значений, а не через обертку `List.of` или `Arrays.asList`.

Библиотек в Java невероятно много, и каждая имеет свои фишки.

*Если приводить другие примеры, то в голову сразу приходит библиотека **Jackson**, которая помогает Spring творить магию, превращая ваши объекты в JSON.*

Изучайте библиотеки и, скорее всего, они помогут вам избежать огромного количества рутинной работы, ведь авторы библиотек уже написали эту рутину за вас.